

8 Strings

Figure 8.1 credit: modification of work "Project 366 #65: 050316 A Night On The Tiles", by Pete/Flickr, CC BY 2.0

Chapter Outline

8.1 String operations
8.2 String slicing
8.3 Searching/testing strings
8.4 String formatting
8.5 Splitting/joining strings
8.6 Chapter summary

Introduction

A **string** is a sequence of characters. Python provides useful methods for processing string values. In this chapter, string methods will be demonstrated including comparing string values, string slicing, searching, testing, formatting, and modifying.

8.1 String operations

Learning objectives

By the end of this section you should be able to

- Compare strings using logical and membership operators.
- Use `lower()` and `upper()` string methods to convert string values to lowercase and uppercase characters.

String comparison

String values can be compared using logical operators (<, <=, >, >=, ==, !=) and membership operators (`in` and `not in`). When comparing two string values, the matching characters in two string values are compared sequentially until a decision is reached. For comparing two characters, ASCII values are used to apply logical operators.

Operator	Description	Example	Output	Explanation
> or >=	Checks whether the first string value is greater than (or greater than or equal to) the second string value.	`"c" > "d"`	`False`	When comparing `"c"` operand to `"d"` operand, the ASCII value for `"c"` is smaller than the ASCII value for `"d"`. Therefore, `"c" < "d"`. The expression `"c" > "d"` evaluates to False.
< or <=	Checks whether the first string value is less than (or less than or equal to) the second string value.	`"ab" < "ac"`	`True`	When comparing `"ab"` operand to `"ac"` operand, the first characters are the same, but the second character of `"ab"` is less than the second character in `"ac"` and as such `"ab" < "ac"`.
==	Checks whether two string values are equal.	`"aa" == "aa"`	`True`	Since all characters in the first operand and the second operand are the same, the two string values are equal.
!=	Checks whether two string values are not equal.	`"a" != "b"`	`True`	The two operands contain different string values (`"a"` vs. `"b"`), and the result of checking whether the two are not the same evaluates to True.
`in`	Checks whether the second operand contains the first operand.	`"a" in "bc"`	`False`	Since string `"bc"` does not contain string `"a"`, the output of `"a" in "bc"` evaluates to False.
`not in`	Checks whether the second operand does not contain the first operand.	`"a" not in "bc"`	`True`	Since string `"bc"` does not contain string `"a"`, the output of `"a" not in "bc"` evaluates to True.

Table 8.1 Comparing string values.

CONCEPTS IN PRACTICE

Using logical and membership operators to compare string values

1. What is the output of (`"aaa" < "aab"`)?
   a. `True`
   b. `False`
2. What is the output of (`"aa" < "a"`)?
   a. `True`
   b. `False`
3. What is the output of (`"aples" in "apples"`)?

a. undefined
b. True
c. False

lower() and upper()

Python has many useful methods for modifying strings, two of which are `lower()` and `upper()` methods. The **lower()** method returns the converted alphabetical characters to lowercase, and the **upper()** method returns the converted alphabetical characters to uppercase. Both the `lower()` and `upper()` methods do *not* modify the string.

EXAMPLE 8.1

Converting characters in a string

In the example below, the `lower()` and `upper()` string methods are called on the string variable x to convert all characters to lowercase and uppercase, respectively.

```
x = "Apples"

# The lower() method converts a string to all lowercase characters
print(x.lower())

# The upper() method converts a string to all uppercase characters
print(x.upper())
```

The above code's output is:

```
apples
APPLES
```

CONCEPTS IN PRACTICE

Using lower() and upper()

4. What is the output of `"aBbA".lower()`?
   a. abba
   b. ABBA
   c. abbA
5. What is the output of `"aBbA".upper()`?
   a. abba
   b. ABBA
   c. ABbA
   d. aBBA
6. What is the output of `("a".upper() == "A")`?

a. `True`
b. `False`

TRY IT

Number of characters in the string

A string variable, `s_input`, is defined. Use `lower()` and `upper()` to convert the string to lowercase and uppercase, and print the results in the output. Also, print the number of characters in the string, including space characters.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/8-1-string-operations)

TRY IT

What is my character?

Given the string, `s_input`, which is a one-character string object, if the character is between `"a"` and `"t"` or `"A"` and `"T"`, print `True`. Otherwise, print `False`.
Hint: You can convert `s_input` to lowercase and check if `s_input` is between `"a"` and `"t"`.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/8-1-string-operations)

8.2 String slicing

Learning objectives

By the end of this section you should be able to

- Use string indexing to access characters in the string.
- Use string slicing to get a substring from a string.
- Identify immutability characteristics of strings.

String indexing

A string is a type of sequence. A string is made up of a sequence of characters indexed from left to right, starting at 0. For a string variable `s`, the left-most character is indexed 0 and the right-most character is indexed `len(s) - 1`. Ex: The length of the string `"Cloud"` is 5, so the last index is 4.

Negative indexing can also be used to refer to characters from right to left starting at -1. For a string variable `s`, the left-most character is indexed `-len(s)` and the right-most character is indexed -1. Ex: The length of the string `"flower"` is 6, so the index of the first character with negative indexing is -6.

CHECKPOINT

String indexing

String indexing; ch 8, video 1 (https://www.openstax.org/r/string-indexing)

CONCEPTS IN PRACTICE

Accessing characters in a string using indexing

1. Which character is at index 1 in the string `"hello"`?
   a. `"h"`
   b. `"e"`
   c. `"o"`
2. What is the character at index -2 in the string `"Blue"`?
   a. `"e"`
   b. `"u"`
   c. `"l"`
3. What is the output of the following code?

```
word = "chance"
print(word[-1] == word[5])
```

   a. `True`
   b. `False`

String slicing

String slicing is used when a programmer must get access to a sequence of characters. Here, a string slicing operator can be used. When `[a:b]` is used with the name of a string variable, a sequence of characters starting from index a (inclusive) up to index b (exclusive) is returned. Both a and b are optional. If a or b are not provided, the default values are 0 and `len(string)`, respectively.

EXAMPLE 8.2

Getting the minutes

Consider a time value is given as `"hh:mm"` with `"hh"` representing the hour and `"mm"` representing the minutes. To retrieve only the string's minutes portion, the following code can be used:

```
time_string = "13:46"
minutes = time_string[3:5]
print(minutes)
```

The above code's output is:

```
46
```

EXAMPLE 8.3

Getting the hour

Consider a time value is given as `"hh:mm"` with `"hh"` representing the hour and `"mm"` representing the minutes. To retrieve only the string's hour portion, the following code can be used:

```
time_string = "14:50"
hour = time_string[:2]
print(hour)
```

The above code's output is:

```
14
```

CONCEPTS IN PRACTICE

Getting a substring using string slicing

4. What is the output of the following code?

   ```
   a_string = "Hello world"
   print(a_string[2:4])
   ```

   a. `"el"`
   b. `"ll"`
   c. `"llo"`

5. What is the output of the following code?

   ```
   location = "classroom"
   print(location[-3:-1])
   ```

   a. `"ro"`
   b. `"oo"`
   c. `"oom"`

6. What is the output of the following code?

   ```
   greeting = "hi Leila"
   name = greeting[3:]
   ```

   a. `" Leila"`
   b. `"Leila"`
   c. `"ila"`

String immutability

String objects are **immutable** meaning that string objects cannot be modified or changed once created. Once

a string object is created, the string's contents cannot be altered by directly modifying individual characters or elements within the string. Instead, to make changes to a string, a new string object with the desired changes is created, leaving the original string unchanged.

CHECKPOINT

Strings are immutable

Strings are immutable; ch 8, video 2 (https://www.openstax.org/r/strings-are-immutable)

CONCEPTS IN PRACTICE

Modifying string content

7. What is the correct way of replacing the first character in a string to character "*" in a new string?
   a.
   ```
   x = "string"
   x[0] = "*"
   ```
   b.
   ```
   x = "string"
   x = "*" + x[1:]
   ```
   c.
   ```
   x = "string"
   x = "*" + x
   ```
8. What type of error will result from the following code?

   ```
   string_variable = "example"
   string_variable[-1] = ""
   ```

   a. `TypeError`
   b. `IndexError`
   c. `NameError`
9. What is the output of the following code?

   ```
   str = "morning"
   str = str[1]
   print(str)
   ```

   a. `TypeError`
   b. `m`
   c. `o`

TRY IT

Changing the greeting message

Given the string `"Hello my fellow classmates"` containing a greeting message, print the first word by getting the beginning of the string up to (and including) the 5th character. Change the first word in the string to `"Hi"` instead of `"hello"` and print the greeting message again.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/

8-2-string-slicing)

TRY IT

Editing the string at specified locations

Given a string variable, `string_variable`, and a list of indexes, remove characters at the specified indexes and print the resulting string.

```
Input:
string_variable = "great"
indices = [0, 1]


Prints:
eat
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/8-2-string-slicing)

8.3 Searching/testing strings

Learning objectives

By the end of this section you should be able to

- Use the `in` operator to identify whether a given string contains a substring.
- Call the `count()` method to count the number of substrings in a given string.
- Search a string to find a substring using the `find()` method.
- Use the `index()` method to find the index of the first occurrence of a substring in a given string.
- Write a for loop on strings using in operator.

in operator

The **in** Boolean operator can be used to check if a string contains another string. `in` returns `True` if the first string exists in the second string, `False` otherwise.

CHECKPOINT

What is in the phrase?

What is the phrase?; ch 8, video 3 (https://www.openstax.org/r/what-in-phrase)

CONCEPTS IN PRACTICE

Using in operator to find substrings

1. What is the output of `("a" in "an umbrella")`?
   a. `2`

b. `False`
c. `True`
d. `1`

2. What is the output of `("ab" in "arbitrary")`?
   a. `True`
   b. `False`
3. What is the output of `("" in "string")`?
   a. `True`
   b. `False`

For loop using in operator

The `in` operator can be used to iterate over characters in a string using a `for` loop. In each `for` loop iteration, one character is read and will be the loop variable for that iteration.

CHECKPOINT

for loop using in operator

For loop using in operator; ch 8, video 4 (https://www.openstax.org/r/for-loop-in-operator)

CONCEPTS IN PRACTICE

Using in operator within for loop

4. What is the output of the following code?

```
for c in "string":
  print(c, end = "")
```

   a. `string`
   b. `s`
   `t`
   `r`
   `i`
   `n`
   `g`
   c. `s t r i n g`

5. What is the output of the following code?

```
count = 0
for c in "abca":
  if c == "a":
    count += 1
print(count)
```

   a. `0`

b. 1
c. 2

6. What is the output of the following code?

```
word = "cab"
for i in word:
  if i == "a":
    print("A", end = "")
  if i == "b":
    print("B", end = "")
  if i == "c":
    print("C", end = "")
```

a. cab
b. abc
c. CAB
d. ABC

count()

The **count()** method counts the number of occurrences of a substring in a given string. If the given substring does not exist in the given string, the value `0` is returned.

CHECKPOINT

Counting the number of occurrences of a substring

Counting the number of occurrences of a substring; ch 8, video 5 (https://www.openstax.org/r/counting-substrings)

CONCEPTS IN PRACTICE

Using count() to count the number of substrings

7. What is the output of (`aaa".count("a")`)?
   a. True
   b. 1
   c. 3
8. What is the output of (`"weather".count("b")`)?
   a. 0
   b. -1
   c. False
9. What is the output of (`"aaa".count("aa")`)?
   a. 1
   b. 2
   c. 3

find()

The **find()** method returns the index of the first occurrence of a substring in a given string. If the substring does not exist in the given string, the value of `-1` is returned.

CHECKPOINT

Finding the first index of a substring

Finding the first index of a substring; ch 8, video 6 (https://www.openstax.org/r/finding-substring-index)

CONCEPTS IN PRACTICE

Using find() to locate a substring

10. What is the output of `"banana".find("a")`?

- **a.** `1`
- **b.** `3`
- **c.** `5`

11. What is the output of `"banana".find("c")`?

- **a.** `0`
- **b.** `-1`
- **c.** `ValueError`

12. What is the output of `"b".find("banana")`?

- **a.** `-1`
- **b.** `0`
- **c.** `ValueError`

index()

The **index()** method performs similarly to the `find()` method in which the method returns the index of the first occurrence of a substring in a given string. The `index()` method assumes that the substring exists in the given string; otherwise, throws a `ValueError`.

EXAMPLE 8.4

Getting the time's minute portion

Consider a time value is given as part of a string using the format of `"hh:mm"` with `"hh"` representing the hour and `"mm"` representing the minutes. To retrieve only the string's minute portion, the following code can be used:

```
time_string = "The time is 12:50"
index = time_string.index(":")
print(time_string[index+1:index+3])
```

The above code's output is:

```
50
```

CONCEPTS IN PRACTICE

Using index() to locate a substring

13. What is the output of `"school".index("o")`?

a. `3`
b. `4`
c. `-3`

14. What is the output of `"school".index("ooo")`?

a. `3`
b. `4`
c. `ValueError`

15. What is the output of the following code?

```
sentence = "This is a sentence"
index = sentence.index(" ")
print(sentence[:index])
```

a. `"This"`
b. `"This "`
c. `"sentence"`

TRY IT

Finding all spaces

Write a program that, given a string, counts the number of space characters in the string. Also, print the given string with all spaces removed.

```
Input: "This is great"


Prints:
2
Thisisgreat
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/8-3-searching-testing-strings)

8.4 String formatting

Learning objectives

By the end of this section you should be able to

- Format a string template using input arguments.
- Use `format()` to generate numerical formats based on a given template.

String format specification

Python provides string substitutions syntax for formatting strings with input arguments. **Formatting string** includes specifying string pattern rules and modifying the string according to the formatting specification. Examples of formatting strings include using patterns for building different string values and specifying modification rules for the string's length and alignment.

String formatting with replacement fields

Replacement fields are used to define a pattern for creating multiple string values that comply with a given format. The example below shows two string values that use the same template for making requests to different individuals for taking different courses.

EXAMPLE 8.5

String values from the same template

```
Dear John, I'd like to take a programming course with Prof. Potter.

Dear Kishwar, I'd like to take a math course with Prof. Robinson.
```

In the example above, replacement fields are 1) the name of the individual the request is being made to, 2) title of the course, and 3) the name of the instructor. To create a template, replacement fields can be added with {} to show a placeholder for user input. The **format()** method is used to pass inputs for replacement fields in a string template.

EXAMPLE 8.6

String template formatting for course enrollment requests

A string template with replacement fields is defined below to create string values with different input arguments. The `format()` method is used to pass inputs to the template in the same order.

```
s = "Dear {}, I'd like to take a {} course with Prof. {}."

print(s)
print(s.format("John", "programming", "Potter"))
print(s.format("Kishwar", "math", "Robinson"))
```

The above code's output is:

```
Dear {}, I'd like to take a {} course with Prof. {}.
Dear John, I'd like to take a programming course with Prof. Potter.
Dear Kishwar, I'd like to take a math course with Prof. Robinson.
```

CONCEPTS IN PRACTICE

String template and formatting

1. What is the output of `print("Hello {}!".format("Ana"))`?
   a. `Ana`
   b. `Hello Ana`
   c. `Hello Ana!`
2. What is the output of `print("{}:{}".format("One", "1"))`?
   a. `One1`
   b. `One:1`
   c. `1:One`
3. What is the output of `print("{}".format("one", "two", "three"))`?
   a. `one`
   b. `two`
   c. `onetwothree`

Named replacement fields

Replacement fields can be tagged with a label, called **named replacement fields**, for ease of access and code readability. The example below illustrates how named replacement fields can be used in string templates.

EXAMPLE 8.7

Season weather template using named replacement fields

A named replacement argument is a convenient way of assigning name tags to replacement fields and passing values associated with replacement fields using corresponding names (instead of passing values in order).

```
s = "Weather in {season} is {temperature}."

print(s)
print(s.format(season = "summer", temperature = "hot"))
print(s.format(season = "winter", temperature = "cold"))
```

The above code's output is:

```
Weather in {season} is {temperature}.
```

```
Weather in summer is hot.
Weather in winter is cold.
```

MULTIPLE USE OF A NAMED ARGUMENT

Since named replacement fields are referred to using a name key, a named replacement field can appear and be used more than once in the template. Also, positional ordering is not necessary when named replacement fields are used.

```
s = "Weather in {season} is {temperature}; very very {temperature}."

print(s)
print(s.format(season = "summer", temperature = "hot"))
print(s.format(temperature = "cold", season = "winter"))
```

The above code's output is:

```
Weather in {season} is {temperature}; very very {temperature}.
Weather in summer is hot; very very hot.
Weather in winter is cold; very very cold.
```

CONCEPTS IN PRACTICE

Named replacement field examples

4. What is the output of `print("Hey {name}!".format(name = "Bengio"))`?
   a. `Hey name!`
   b. `Hey Bengio`
   c. `Hey Bengio!`
5. What is the output of `print("Hey {name}!".format("Bengio"))`?
   a. `Hey name!`
   b. `KeyError`
   c. `Hey Bengio!`
6. What is the output of the following code?

   ```
   greeting = "Hi"
   name = "Jess"
   print("{greeting} {name}".format(greeting = greeting, name = name))
   ```

   a. `greeting name`
   b. `Hi Jess`
   c. `Jess Hi`

Numbered replacement fields

Python's string `format()` method can use positional ordering to match the numbered arguments. The replacement fields that use the positional ordering of arguments are called **numbered replacement fields**. The indexing of the arguments starts from 0. Ex: `print("{1}{0}".format("Home", "Welcome"))` outputs the string value `"Welcome Home"` as the first argument. `"Home"` is at index 0, and the second argument, `"Welcome"`, is at index 1. Replacing these arguments in the order of `"{1}{0}"` creates the string `"Welcome Home"`.

Numbered replacement fields can use argument's values for multiple replacement fields by using the same argument index. The example below illustrates how an argument is used for more than one numbered replacement field.

EXAMPLE 8.8

Numbered replacement field to build a phrase

Numbered replacement fields are used in this example to build phrases like `"very very cold"` or `"very hot"`.

```
template1 = "{0} {0} {1}"
template2 = "{0} {1}"

print(template1.format("very", "cold"))
print(template2.format("very", "hot"))
```

The above code's output is:

```
very very cold
very hot
```

String length and alignment formatting

Formatting the string length may be needed for standardizing the output style when multiple string values of the same context are being created and printed. The example below shows a use case of string formatting in printing a table with minimum-length columns and specific alignment.

EXAMPLE 8.9

A formatted table of a class roster

A formatted table of a class roster

```
Student Name              Major          Grade
-----------------------------------------------
Manoj Sara          Computer Science        A-
Gabriel Wang     Electrical Engineering      A
Alex Narayanan       Social Sciences        A+
```

In the example above, the table is formatted into three columns. The first column takes up 15 characters and is left-aligned. The second column uses 25 characters and is center-aligned, and the last column uses two characters and is right aligned. Alignment and length format specifications controls are used to create the formatted table.

The field width in string format specification is used to specify the minimum length of the given string. If the string is shorter than the given minimum length, the string will be padded by space characters. A **field width** is included in the format specification field using an integer after a colon. Ex: `{name:15}` specifies that the minimum length of the string values that are passed to the `name` field is 15.

Since the field width can be used to specify the minimum length of a string, the string can be padded with space characters from right, left, or both to be left-aligned, right-aligned, and centered, respectively. The **string alignment type** is specified using <, >, or ^characters after the colon when field length is specified. Ex: `{name:^20}` specifies a named replacement field with the minimum length of 20 characters that is center-aligned.

Alignment Type	Symbol	Example	Output
Left-aligned	<	`template = "{hex:<7}{name:<10}"` `print(template.format(hex = "#FF0000", name = "Red")) print(template.format(hex = "#00FF00", name = "green"))`	`#FF0000Red` `#00FF00green`
Right-aligned	>	`template = "{hex:>7}{name:>10}"` `print(template.format(hex = "#FF0000", name = "Red")) print(template.format(hex = "#00FF00", name = "green"))`	`#FF0000 Red` `#00FF00 green`
Centered	^	`template = "{hex:^7}{name:^10}"` `print(template.format(hex = "#FF0000", name = "Red")) print(template.format(hex = "#00FF00", name = "green"))`	`#FF0000 Red` `#00FF00 green`

Table 8.2 String alignment formatting.

CONCEPTS IN PRACTICE

Specifying field width and alignment

7. What is the output of the following code?

```
template = "{name:12}"
formatted_name = template.format(name = "Alice")
print(len(formatted_name))
```

a. 5
b. 12

c. `"Alice"`

8. What is the output of the following code?

```
template = "{greeting:>6}"
formatted_greeting = template.format(greeting = "Hello")
print(formatted_greeting[0])
```

a. H
b. `" Hello"`
c. Space character

9. What is the output of the following code?

```
template = "{:5}"
print(template.format("123456789"))
```

a. `56789`
b. `123456`
c. `123456789`

Formatting numbers

The `format()` method can be used to format numerical values. Numerical values can be padded to have a given minimum length, precision, and sign character. The syntax for modifying numeric values follows the `{[index]:[width][.precision][type]}` structure. In the given syntax,

- The `index` field refers to the index of the argument.
- The `width` field refers to the minimum length of the string.
- The `precision` field refers to the floating-point precision of the given number.
- The `type` field shows the type of the input that is passed to the `format()` method. Floating-point and decimal inputs are identified by `"f"` and `"d"`, respectively. String values are also identified by `"s"`.

The table below summarizes formatting options for modifying numeric values.

Example	Output	Explanation
`print("{:.7f}".format(0.9795))`	`0.9795000`	The format specification `.7` shows the output must have seven decimal places. The `f` specification is an identifier of floating-point formatting.
`print("{:.3f}".format(12))`	`12.000`	The format specification `.3` shows the output must have three decimal places. The `f` specification is an identifier of floating-point formatting.

Table 8.3 Numerical formatting options.

Example	Output	Explanation
`print("{:+.2f}".format(4))`	`+4.00`	The format specification `.2` shows the output must have two decimal places. The `f` specification is an identifier of floating-point formatting. The + sign before the precision specification adds a sign character to the output.
`print("{:0>5d}".format(5))`	`00005`	The format specification `0>5` defines the width field as 5, and thus the output must have a minimum length of 5. And, if the number has fewer than five digits, the number must be padded with 0's from the left side. The d specification is an identifier of a decimal number formatting.
`print("{:.3s}".format("12.50"))`	`12.`	The format specification `.3` shows the output will have three characters. The s specification is an identifier of string formatting.

Table 8.3 Numerical formatting options.

CONCEPTS IN PRACTICE

Numeric value formatting examples

10. What is the output of `print('{0:.3f}'.format(3.141592))`?

a. `3.141592`
b. `3.1`
c. `3.142`

11. What is the output of `print('{:1>3d}'.format(3))`?

a. `113`
b. `311`
c. `3.000`

12. What is the output of `print('{:+d}'.format(123))`?

a. `123`
b. `+123`
c. `:+123`

TRY IT

Formatting a list of numbers

Given a list of numbers (floating-point or integer), print numbers with two decimal place precision and at

least six characters.

```
Input: [12.5, 2]


Prints: 012.50

002:00
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/8-4-string-formatting)

8.5 Splitting/joining strings

Learning objectives

By the end of this section you should be able to

- Use the `split()` method to split a string into substrings.
- Combine objects in a list into a string using `join()` method.

split()

A string in Python can be broken into substrings given a **delimiter**. A delimiter is also referred to as a separator. The **split()** method, when applied to a string, splits the string into substrings by using the given argument as a delimiter. Ex: `"1-2".split('-')` returns a list of substrings `["1", "2"]`. When no arguments are given to the `split()` method, blank space characters are used as delimiters. Ex: `"1\t2\n3 4".split()` returns `["1", "2", "3", "4"]`.

CHECKPOINT

split() for breaking down the string into tokens

split() for breaking down the string into tokens; ch 8, video 7 (https://www.openstax.org/r/split-function)

CONCEPTS IN PRACTICE

Examples of string delimiters and split() method

1. What is the output of `print("1*2*3*".split('*'))`?
   a. `["1", "*", "2", "*", "3", "*"]`
   b. `["1", "2', "3"]`
   c. `[1, 2, 3]`
2. What is the output of `print("a year includes 12 months".split())`?
   a. `["a year includes 12 months"]`
   b. `["a", "year", "includes", 12, "months"]`
   c. `["a", "year", "includes", "12", "months"]`
3. What is the output of the following code?

```
s = """This is a test"""

out = s.split()
print(out)
```

a. Error
b. ['This', 'is', 'a', 'test']
c. >['This', 'is a', 'test']

join()

The **join()** method is the inverse of the `split()` method: a list of string values are concatenated together to form one output string. When joining string elements in the list, the delimiter is added in-between elements. Ex: `','.join(["this", "is", "great"])` returns `"this,is,great"`.

CHECKPOINT

join() for combining tokens into one string

join() for combing tokens into one string; ch 8, video 8 (https://www.openstax.org/r/join-function)

CONCEPTS IN PRACTICE

Applying join() method on list of string values

4. What is the output of the following code?

```
elements = ['A', 'beautiful', 'day', 'for', 'learning']

print(",".join(elements))
```

a. 'A beautiful day for learning'
b. ['A, beautiful, day, for, learning']
c. 'A,beautiful,day,for,learning'

5. What is the length of the string `"sss".join(["1","2"])`?
a. 2
b. 5
c. 8

6. What is the value stored in the variable out?

```
s = ["1", "2"]
out = "".join(s)
```

a. 12
b. "12"
c. "1 2"

TRY IT

Unique and comma-separated words

Write a program that accepts a comma-separated sequence of words as input, and prints words in separate lines. Ex: Given the string `"happy,smiling,face"`, the output would be:

```
happy
smiling
face
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/8-5-splitting-joining-strings)

TRY IT

Lunch order

Use the `join()` method to repeat back a user's order at a restaurant, separated by commas. The user will input each food item on a separate line. When finished ordering, the user will enter a blank line. The output depends on how many items the user orders:

- If the user inputs nothing, the program outputs:
  `You ordered nothing`.
- If the user inputs one item (Ex: eggs), the program outputs:
  `You ordered eggs`.
- If the user inputs two items (Ex: eggs, ham), the program outputs:
  `You ordered eggs and ham`.
- If the user inputs three or more items (Ex: eggs, ham, toast), the program outputs:
  `You ordered eggs, ham, and toast`.

In the general case with three or more items, each item should be separated by a comma and a space. The word "and" should be added before the last item.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/8-5-splitting-joining-strings)

8.6 Chapter summary

Highlights from this chapter include:

- A string is a sequence of characters.
- Logical operators can be used to compare two string values. String comparison is done by comparing corresponding ASCII values of characters in the order of appearance in the string.
- String indexing is used to access a character or a sequence of characters in the string.
- String objects are immutable.
- String splicing.

At this point, you should be able to write programs dealing with string values.

Method	Description
`len()`	Returns the string length.
`upper()`	Returns uppercase characters.
`lower()`	Returns lowercase characters.
`count()`	Returns the number of a given substring in a string.
`find()`	Returns the index of the first occurrence of a given substring in a string. If the substring does not exist in the string, -1 is returned.
`index()`	Returns the index of the first occurrence of a given substring in a string. If the substring does not exist in the string, a `ValueError` is returned.
`format()`	Used to create strings with specified patterns using arguments.
`join()`	Takes a list of string values and combines string values into one string by placing a given separator between values.
`split()`	Separates a string into tokens based on a given separator string. If no separator string is provided, blank space characters are used as separators.
Operator	**Description**
`in`	Checks if a substring exists in a string.
`in` operator in a `for` loop	`for character in string:` `  # loop body`

Table 8.4 Chapter 8 reference.